

Mathan M - Django Portfolio Website

Responsive personal portfolio for Frontend, Backend and API Development

This project is an original Django portfolio inspired by your supplied green and yellow layout. It includes a responsive hero section, services, About Me, tool/skill cards, project cards, education/work journey, and a contact form. The contact form saves messages in the Django database.

Feature	What it does
Responsive layout	Works on mobile, tablet and desktop.
Django admin	Add Services, Skills and Projects from /admin/.
Contact form	Stores visitor messages in SQLite database.
Portfolio content	Uses your web developer profile: frontend, Django backend, SQL and APIs.
No external images	The visual sections use original CSS illustrations, so the project runs immediately.

Project Folder Structure

```
Mathan_Portfolio_Django_Full/
|-- config/                # Django project settings and URLs
|-- portfolio/
|   |-- templates/portfolio/home.html
|   |-- static/portfolio/css/style.css
|   |-- static/portfolio/js/main.js
|   |-- models.py          # Service, Skill, Project, ContactMessage
|   |-- views.py           # Homepage + contact form
|   |-- admin.py           # Admin panel configuration
|-- requirements.txt
|-- manage.py
`-- README.md
```

Run the Project in VS Code

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Then open **http://127.0.0.1:8000/**. To add your projects and skills, open **http://127.0.0.1:8000/admin/**.

Main Django Code

1. Database models - portfolio/models.py

```
class Project(models.Model):
    title = models.CharField(max_length=150)
    category = models.CharField(max_length=120)
    description = models.TextField()
    tech_stack = models.CharField(max_length=250)
    image = models.ImageField(upload_to="projects/", blank=True, null=True)
    order = models.PositiveIntegerField(default=0)

class ContactMessage(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    phone = models.CharField(max_length=25, blank=True)
    interest = models.CharField(max_length=100, blank=True)
    message = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

2. Homepage view - portfolio/views.py

```
def home(request):
    if request.method == "POST":
        form = ContactForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, "Thanks! Your message has been sent.")
            return redirect("home")
    else:
        form = ContactForm()

    return render(request, "portfolio/home.html", {
        "form": form,
        "services": list(Service.objects.all()) or DEFAULT_SERVICES,
        "skills": list(Skill.objects.all()) or DEFAULT_SKILLS,
        "projects": list(Project.objects.all()) or DEFAULT_PROJECTS,
    })
```

3. Route - config/urls.py

```
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", views.home, name="home"),
]
```

How to Customize

Your title and introduction: edit portfolio/templates/portfolio/home.html.

Colors and responsive design: edit portfolio/static/portfolio/css/style.css.

Projects, skills and services: create a superuser, open Django Admin, and add items. The homepage shows default sample items until you add your own.

Contact details: replace the phone number and email in the contact section of home.html.

Admin and Deployment Guide

Add New Projects from Django Admin

1. Run: `python manage.py createsuperuser`
2. Create a username, email and password.
3. Run the server and open `/admin/`.
4. Choose **Projects** -> **Add Project**.
5. Enter title, category, description, tech stack, optional image and order number.
6. Save. The new project will show on the homepage.

Deploy Later

For production, set `DEBUG = False`, use a new private `SECRET_KEY`, add your hosting domain to `ALLOWED_HOSTS`, install Gunicorn and WhiteNoise, then deploy to a Django-compatible host such as Render or Railway. Use PostgreSQL for production instead of the local SQLite database.

Important Files

File	Use
<code>portfolio/templates/portfolio/home.html</code>	Portfolio page design and text
<code>portfolio/static/portfolio/css/style.css</code>	All design colors, layout and responsive style
<code>portfolio/models.py</code>	Database models for content and contact messages
<code>portfolio/admin.py</code>	Manage content inside Django Admin
<code>README.md</code>	Quick run instructions

The ZIP file contains the complete runnable Django project and this guide explains the main files and commands.